

AI-TECH

Przykład 1 DO LABORATORIUM 3

z UCZENIA MASZYNOWEGO

Klasyfikacja nienadzorowana

Adam Kurowski

Klasyfikacja nadzorowana i nienadzorowana bazująca na metrykach odległości

Rzeczpospolita
PolskaUnia Europejska
Europejski Fundusz
Rozwoju Regionalnego

Projekt współfinansowany ze środków Unii Europejskiej w ramach Europejskiego Funduszu Rozwoju Regionalnego Program Operacyjny Polska Cyfrowa na lata 2014-2020, Oś Priorytetowa nr 3 "Cyfrowe kompetencje społeczeństwa" Działanie nr 3.2 "Innowacyjne rozwiązania na rzecz aktywizacji cyfrowej" Tytuł projektu: „Akademia Innowacyjnych Zastosowań Technologii Cyfrowych (AI Tech)”

Przykład 1.: Klasyfikacja nadzorowana i nienadzorowana bazująca na metrykach odległości

Analiza struktury danych wejściowych

Klasyfikacja nienadzorowana to technika przydatna w sytuacji, gdy nie mamy pełnej informacji o strukturze danych, które przetwarzamy. Możemy na przykład jedynie wiedzieć, ile klas danych występuje, jednak nie mamy pełnej informacji o tym, które dokładnie elementy zbioru danych należą do konkretnych klas.

Jest to sytuacja wymagająca specyficznego podejścia do zadania klasyfikacji i podjęcia próby przypisania do poszczególnych klas zbiorów danych wyróżniających się w jakiś specyficzny sposób z ogółu zbioru danych. Jedną z najprostszych technik mogących wykonać tak zdefiniowane zadanie jest algorytm k średnich (ang. k means), który jest co do swojej zasady działania nieco podobny do algorytmu k najbliższych sąsiadów (ang. k nearest neighbours, kNN). Obydwa te algorytmy w swoim najprostszym wariantcie posługują się odległością euklidesową pomiędzy punktami (lub pomiędzy punktami, a środkami klastrów) do zdefiniowania przynależności przetwarzanych punktów danych do konkretnej klasy. W tym zadaniu zapoznamy się z obydwooma algorytmami. Zaczniemy od zagadnienia znanego już z poprzednich ćwiczeń - algorytmu k najbliższych sąsiadów.

```
In [1]: # Import bibliotek
import numpy as np # obliczenia numeryczne
import pandas as pd # przetwarzanie danych "tabelarycznych"
import matplotlib.pyplot as plt # wizualizacja danych
from sklearn.datasets import load_iris # zbiór danych - wykorzystamy słynny zbiór Iris
from sklearn.decomposition import PCA # redukcja wymiarowości
from sklearn.neighbors import KNeighborsClassifier # implementacja algorytmu k najb
```

```
from sklearn.cluster import KMeans # implementacja algorytmu k średnich  
from mlxtend.plotting import plot_decision_regions # wizualizacja obszarów decyzyjnych
```

```
c:\Users\Igor\anaconda3\Lib\site-packages\pandas\core\computation\expressions.py:22:  
UserWarning: Pandas requires version '2.10.2' or newer of 'numexpr' (version '2.10.  
1' currently installed).  
from pandas.core.computation.check import NUMEXPR_INSTALLED
```

```
In [2]: %pip install mlxtend
```

```

Collecting mlxtend
  Using cached mlxtend-0.24.0-py3-none-any.whl.metadata (7.3 kB)
Collecting scipy>=1.16.3 (from mlxtend)
  Downloading scipy-1.17.1-cp313-cp313-win_amd64.whl.metadata (60 kB)
Collecting numpy>=2.3.5 (from mlxtend)
  Downloading numpy-2.4.4-cp313-cp313-win_amd64.whl.metadata (6.6 kB)
Collecting pandas>=2.3.3 (from mlxtend)
  Downloading pandas-3.0.2-cp313-cp313-win_amd64.whl.metadata (19 kB)
Collecting scikit-learn>=1.8.0 (from mlxtend)
  Downloading scikit_learn-1.8.0-cp313-cp313-win_amd64.whl.metadata (11 kB)
Collecting matplotlib>=3.10.8 (from mlxtend)
  Downloading matplotlib-3.10.9-cp313-cp313-win_amd64.whl.metadata (52 kB)
Collecting joblib>=1.5.2 (from mlxtend)
  Using cached joblib-1.5.3-py3-none-any.whl.metadata (5.5 kB)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\igor\anaconda3\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (1.3.1)
Requirement already satisfied: cyclor>=0.10 in c:\users\igor\anaconda3\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\igor\anaconda3\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (4.55.3)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\igor\anaconda3\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (1.4.8)
Requirement already satisfied: packaging>=20.0 in c:\users\igor\anaconda3\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (24.2)
Requirement already satisfied: pillow>=8 in c:\users\igor\anaconda3\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (11.1.0)
Requirement already satisfied: pyparsing>=3 in c:\users\igor\anaconda3\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (3.2.0)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\igor\anaconda3\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (2.9.0.post0)
Requirement already satisfied: tzdata in c:\users\igor\anaconda3\lib\site-packages (from pandas>=2.3.3->mlxtend) (2025.2)
Requirement already satisfied: six>=1.5 in c:\users\igor\anaconda3\lib\site-packages (from python-dateutil>=2.7->matplotlib>=3.10.8->mlxtend) (1.17.0)
Requirement already satisfied: threadpoolctl>=3.2.0 in c:\users\igor\anaconda3\lib\site-packages (from scikit-learn>=1.8.0->mlxtend) (3.5.0)
Using cached mlxtend-0.24.0-py3-none-any.whl (1.4 MB)
Using cached joblib-1.5.3-py3-none-any.whl (309 kB)
Downloading matplotlib-3.10.9-cp313-cp313-win_amd64.whl (8.2 MB)
----- 0.0/8.2 MB ? eta -:-:-
----- 0.8/8.2 MB 5.4 MB/s eta 0:00:02
----- 2.1/8.2 MB 6.0 MB/s eta 0:00:02
----- 3.7/8.2 MB 6.1 MB/s eta 0:00:01
----- 5.2/8.2 MB 6.6 MB/s eta 0:00:01
----- 7.1/8.2 MB 7.1 MB/s eta 0:00:01
----- 8.2/8.2 MB 7.3 MB/s eta 0:00:00
Downloading numpy-2.4.4-cp313-cp313-win_amd64.whl (12.3 MB)
----- 0.0/12.3 MB ? eta -:-:-
----- 2.1/12.3 MB 10.5 MB/s eta 0:00:01
----- 4.7/12.3 MB 12.0 MB/s eta 0:00:01
----- 7.9/12.3 MB 12.8 MB/s eta 0:00:01
----- 11.0/12.3 MB 13.5 MB/s eta 0:00:01
----- 12.3/12.3 MB 13.2 MB/s eta 0:00:00
Downloading pandas-3.0.2-cp313-cp313-win_amd64.whl (9.7 MB)
----- 0.0/9.7 MB ? eta -:-:-
----- 2.9/9.7 MB 14.0 MB/s eta 0:00:01

```

```

Downloading scipy-1.17.1-cp313-cp313-win_amd64.whl (36.5 MB)
----- 0.0/36.5 MB ? eta -:--:--
-- 1.8/36.5 MB 11.3 MB/s eta 0:00:04
--- 4.5/36.5 MB 11.8 MB/s eta 0:00:03
----- 7.9/36.5 MB 13.3 MB/s eta 0:00:03
----- 11.3/36.5 MB 14.2 MB/s eta 0:00:02
----- 14.4/36.5 MB 14.5 MB/s eta 0:00:02
----- 17.6/36.5 MB 14.7 MB/s eta 0:00:02
----- 19.4/36.5 MB 13.9 MB/s eta 0:00:02
----- 22.3/36.5 MB 14.0 MB/s eta 0:00:02
----- 25.7/36.5 MB 14.3 MB/s eta 0:00:01
----- 28.6/36.5 MB 14.0 MB/s eta 0:00:01
----- 32.2/36.5 MB 14.4 MB/s eta 0:00:01
----- 35.7/36.5 MB 14.6 MB/s eta 0:00:01
----- 36.5/36.5 MB 14.1 MB/s eta 0:00:00

```

```
Attempting uninstall: numpy
```

[illegible]

```
Attempting uninstall: joblib
```

```
Attempting uninstall: scipy
```

7/26

8/26

9/26

10/26

11/26

Successfully installed joblib-1.5.3 matplotlib-3.10.9 mlxtend-0.24.0 numpy-2.4.4 pandas-3.0.2 scikit-learn-1.8.0 scipy-1.17.1

Note: you may need to restart the kernel to use updated packages.

```

WARNING: Failed to remove contents in a temporary directory 'C:\Users\Igor\anaconda3\Lib\site-packages\~umpy'.
You can safely remove it manually.
WARNING: Failed to remove contents in a temporary directory 'C:\Users\Igor\anaconda3\Lib\site-packages\~cipy'.
You can safely remove it manually.
WARNING: Failed to remove contents in a temporary directory 'C:\Users\Igor\anaconda3\Lib\site-packages\~andas'.
You can safely remove it manually.
WARNING: Failed to remove contents in a temporary directory 'C:\Users\Igor\anaconda3\Lib\site-packages\~klearn'.
You can safely remove it manually.
WARNING: Failed to remove contents in a temporary directory 'C:\Users\Igor\anaconda3\Lib\site-packages\~atplotlib'.
You can safely remove it manually.
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the source of the following dependency conflicts.
numba 0.61.0 requires numpy<2.2,>=1.24, but you have numpy 2.4.4 which is incompatible.
sklearn-compat 0.1.3 requires scikit-learn<1.7,>=1.2, but you have scikit-learn 1.8.0 which is incompatible.
streamlit 1.45.1 requires pandas<3,>=1.4.0, but you have pandas 3.0.2 which is incompatible.

```

Do analizy wykorzystamy zbiór danych Iris, możemy łatwo pozyskać go za pomocą API udostępnianego przez bibliotekę scikit-learn.

```

In [2]: # Załadowanie danych z wybranego zbioru danych
dataset_dict = load_iris()

# Wypisanie podstawowych informacji o danych pozyskanych poprzez API biblioteki sci
print('Słownik przechowujący zbiór danych ma następujące klucze:')
print(list(dataset_dict.keys()))

print()
print('Do dyspozycji są następujące cechy:')
print(dataset_dict['feature_names'])

print()
print('Nazwy klas w zbiorze danych:')
print(dataset_dict['target_names'])

# Kilka kluczy ze słownika dla wygody wydzielimy do zmiennych:
data = dataset_dict['data']
target = dataset_dict['target']
target_names = dataset_dict['target_names']

```

Słownik przechowujący zbiór danych ma następujące klucze:

```
['data', 'target', 'frame', 'target_names', 'DESCR', 'feature_names', 'filename', 'data_module']
```

Do dyspozycji są następujące cechy:

```
['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
```

Nazwy klas w zbiorze danych:

```
['setosa' 'versicolor' 'virginica']
```

Dane możemy następnie wstępnie przeanalizować za pomocą szeregu technik. Wiele z nich zaimplementowanych jest w bibliotece Pandas, która zawiera wiele przydatnych narzędzi do pracy z danymi tabelarycznymi. Jest ona także specjalnie przystosowana do radzenia sobie ze zbiorami danych, w których występują brakujące wartości, czy takich zbiorach, dla których konieczna jest np. agregacja danych za okres miesiąca, czy roku. Stąd bardzo często widywana jest ona w skryptach realizujących zaawansowane analizy danych (np. w dziedzinach takich jak tzw. *data science*, czy *big data*).

Na potrzeby naszych analizy utworzymy podstawową strukturę danych wykorzystywaną w bibliotece Pandas, ramkę danych nazywaną DataFrame. Można ją sobie wyobrażać jak logiczną reprezentację arkusza programu takiego jak Microsoft Excel, czy Libre Office Calc. Składa się ona z nazwanych kolumn, w których możemy wpisywać dane.

```
In [4]: # Tworzymy ramkę danych (obiekt DataFrame).
data_df = pd.DataFrame(data)

# Chcemy, żeby kolumny ramki danych posiadały nazwy wskazane przez API.
data_df.columns = dataset_dict['feature_names']

# Dodatkowo dodajemy kolumnę z docelową nazwą klasy.
data_df = data_df.assign(target = target)

# Skonwertowanie int na nazwy z target_names
data_df['target'] = target_names[data_df['target']]

print(data_df)
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	\
0	5.1	3.5	1.4	0.2	
1	4.9	3.0	1.4	0.2	
2	4.7	3.2	1.3	0.2	
3	4.6	3.1	1.5	0.2	
4	5.0	3.6	1.4	0.2	
..	...	...	...	...	
145	6.7	3.0	5.2	2.3	
146	6.3	2.5	5.0	1.9	
147	6.5	3.0	5.2	2.0	
148	6.2	3.4	5.4	2.3	
149	5.9	3.0	5.1	1.8	

	target
0	setosa
1	setosa
2	setosa
3	setosa
4	setosa
..	...
145	virginica
146	virginica
147	virginica
148	virginica
149	virginica

[150 rows x 5 columns]

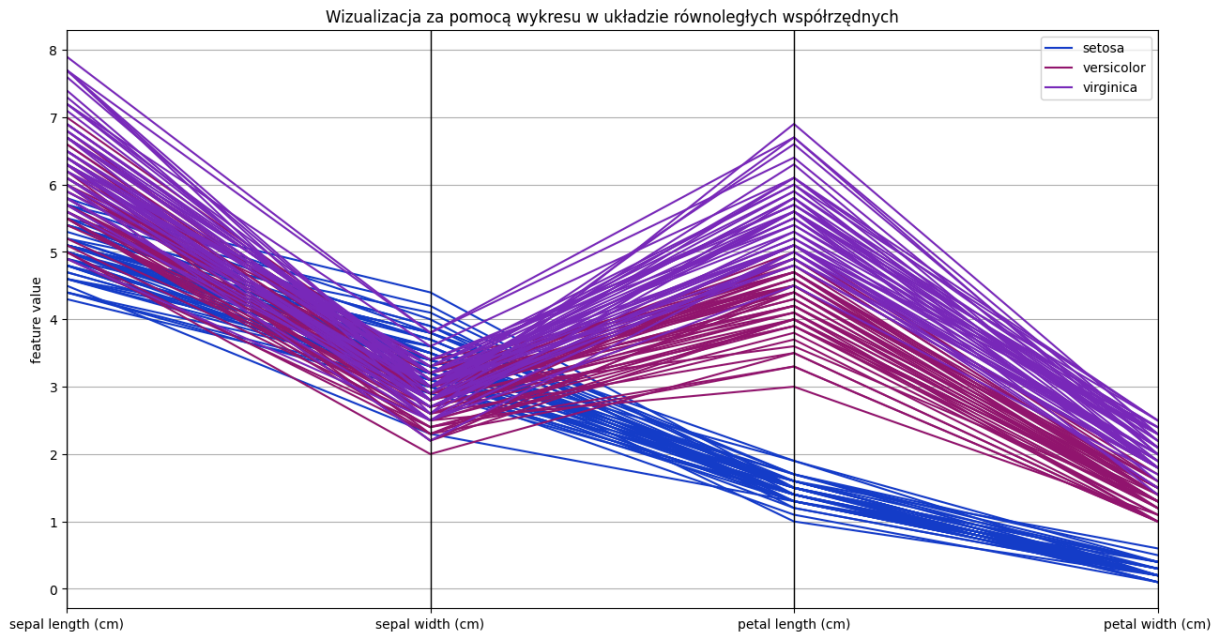
Jednym z najprostszych sposobów jest tzw. wykres współrzędnych równoległych (ang. *parallel coordinates plot*), który polega na przedstawieniu wszystkich wymiarów przetwarzanego zbioru danych w postaci pionowych osi liczbowych. Następnie każdy punkt na takim zestawie osi przedstawiany jest w postaci łamanej, która składa się z wartości danego punktu danych przypisanych do każdej z osi. Działanie takie wykonuje się dla każdego punktu w zbiorze danych, a efekt jest taki, jak po wykonaniu poniższego kodu:

```
In [5]: # Zdefiniowanie rozmiarów wykresu
plt.figure(figsize=(15,8))

# Biblioteka Pandas posiada już gotową funkcję rysującą wizualizację w układzie wsp
pd.plotting.parallel_coordinates(data_df, 'target')

# Na koniec dodajemy opis osi y i tytuł.
plt.ylabel('feature value')
plt.title('Wizualizacja za pomocą wykresu w układzie równoległych współrzędnych')
```

```
Out[5]: Text(0.5, 1.0, 'Wizualizacja za pomocą wykresu w układzie równoległych współrzędny
ch')
```



Zwizualizowanie z pomocą wykresu współrzędnych równoległych pozwala na zorientowanie się, czy występuje separacja pomiędzy poszczególnymi klasami danych i czy jest ona szczególnie zauważalna na którymś z wymiarów zbioru danych.

Dla zbioru Iris taka separacja pomiędzy klasą *setosa* i dwiema pozostałymi klasami jest szczególnie widoczna dla wymiarów *petal length* i *petal width*. Pewna separacja jest także widoczna pomiędzy klasami *versicolor* i *virginica*, jednak tutaj nie jest ona już taka silna i część skrajnych możliwych wartości obu klas nachodzi się na siebie. Najlepsza pod względem separacji między klasami *versicolor* i *virginica* jest kolumna danych *petal length*.

Warto zaznaczyć, że taki sposób wizualizacji pozwala w prosty sposób ocenić separowalność między klasami dla zbiorów danych o wysokiej wymiarowości. Wizualizacja danych np. za pomocą PCA wymaga utraty informacji, tutaj informacja ta jest zachowywana. Tracimy natomiast intuicyjność, gdyż wykres tego typu nie pozwala na ocenienie w prosty sposób odległości pomiędzy punktami.

Wykorzystanie wykresu współrzędnych równoległych jest także przydatne w przypadku danych dla których nie mamy żadnej informacji na temat występujących w nim klas - w przypadku zbioru Iris "rozdzielenie" wykresu sugeruje, że występują w nim przynajmniej dwie klasy obiektów.

Innym wykresem podobnym do wykresu współrzędnych równoległych jest zobrazowanie za pomocą tzw. krzywych Andrewsa lub wykresu Andrewsa. Nie jest on jednak aż tak łatwy do interpretacji. Nadal jednak pozwala ocenić np., czy struktura danych zdradza istnienie odseparowanych od siebie klastrów. Krzywe dla wykresu Andrewsa wylicza się za pomocą następującej zależności:

$$f_{\mathbf{x}}(t) = \frac{x_1}{\sqrt{2}} + x_2 \sin(t) + x_3 \cos(t) + x_4 \sin(2t) + x_5 \cos(2t) + x_6 \sin(3t) + x_7 \cos(3t) + \dots,$$

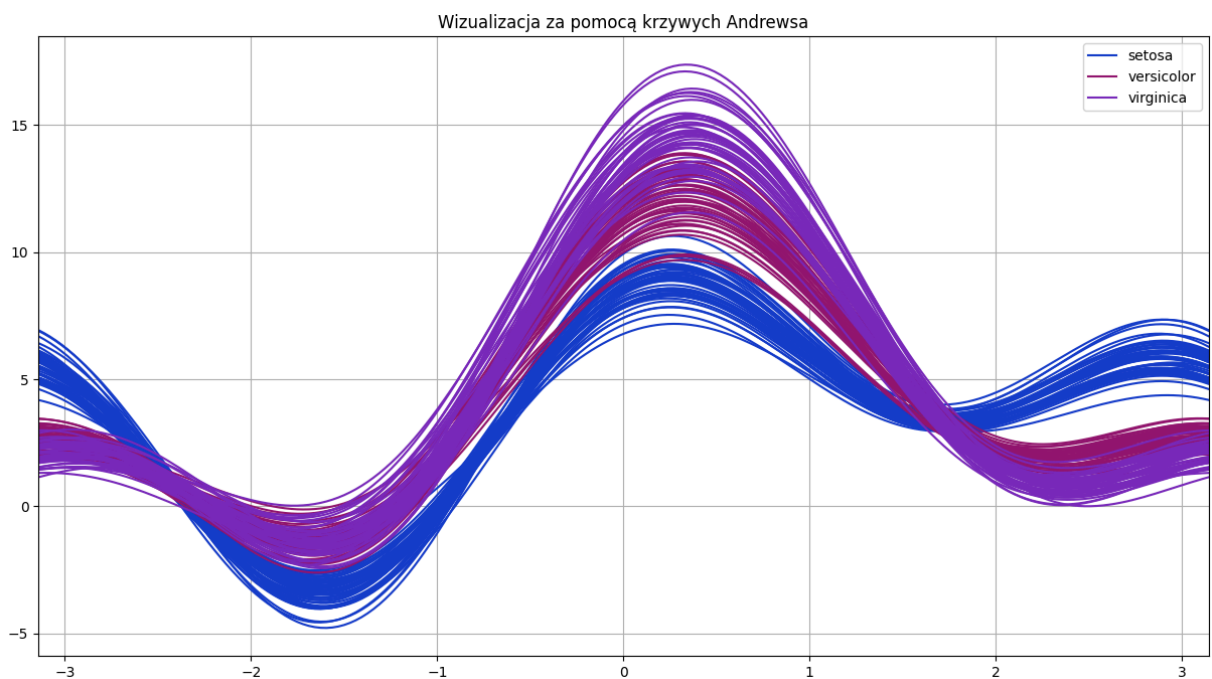
gdzie $\mathbf{x}(t)$ to krzywa odpowiadająca punktowi danych $\mathbf{x} = (x_1, x_2, x_3, \dots, x_n)$, a t jest parametrem, który na wykresie oblicza się dla zakresu wartości $[-\pi; \pi]$. Przykład wykresu krzywych Andrewsa dla zbioru Iris można wygenerować za pomocą kodu poniżej:

```
In [6]: # Zdefiniowanie rozmiarów wykresu
plt.figure(figsize=(15,8))

# Biblioteka Pandas posiada także procedurę generującą wykres Andrewsa:
pd.plotting.andrews_curves(data_df, 'target')

# Dodajemy także tytuł...
plt.title('Wizualizacja za pomocą krzywych Andrewsa')
```

Out[6]: Text(0.5, 1.0, 'Wizualizacja za pomocą krzywych Andrewsa')



W przypadku wykresu krzywych Andrewsa i zbioru Iris także najsilniej odznacza się klasa setosa. Klasy versicolor i virginica są blisko siebie i bez kolorowania klas trudno by było zauważyć że stanowią one dwie odrębne klasy.

Oczywiście, możliwe jest także posłużenie się algorytmami redukcji wymiarowości do analizy struktury danych. Na potrzeby tego ćwiczenia wykorzystamy algorytm PCA i zredukujemy wymiarowość danych do 2 najważniejszych składowych.

```
In [7]: # Obiekt algorytmu redukcji wymiarowości do 2 wymiarów.
pca = PCA(n_components=2)

# Efekt redukcji wymiarowości zapiszemy w osobnej zmiennej:
reduced_data = pca.fit_transform(data)
```

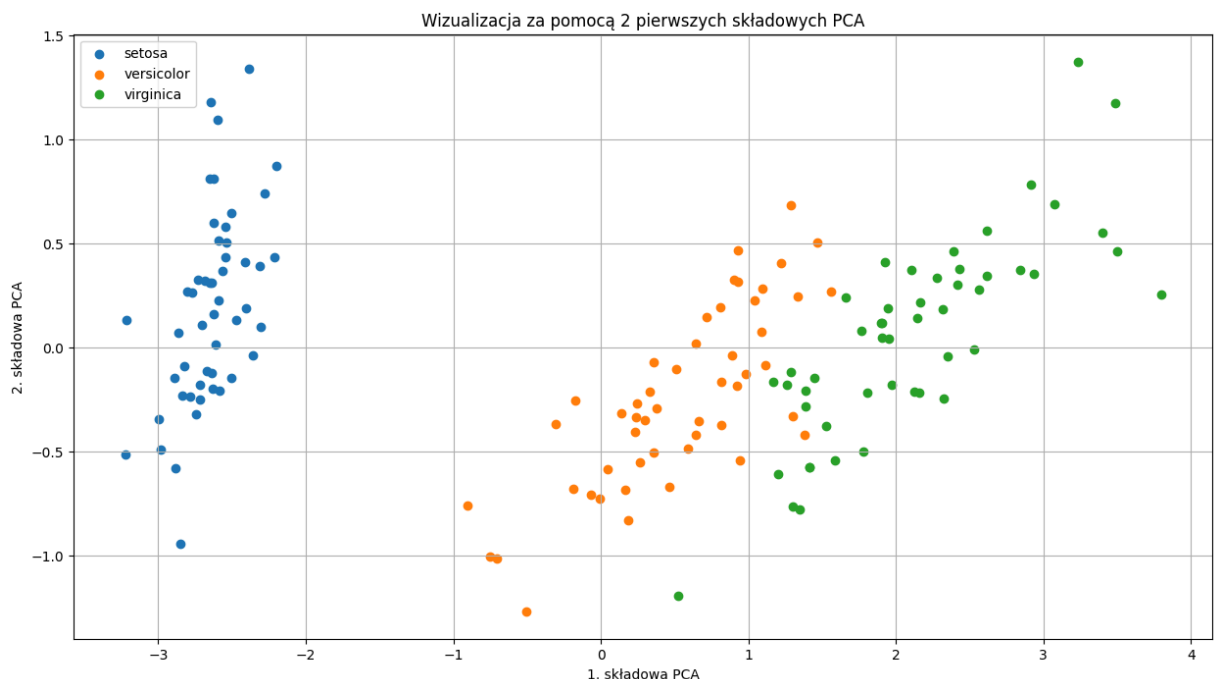
Posiadanie danych w postaci zredukowanej pozwala na ich prostą wizualizację w postaci dwuwymiarowego wykresu punktowego. Należy pamiętać, że każda redukcja wymiarowości wiąże się z utratą części pierwotnej informacji, dlatego należy tego typu wykresy traktować jedynie poglądowo. Wszelkie ostateczne analizy należy prowadzić w miarę możliwości w przestrzeni wysokowymiarowej. Pełnowymiarowej, jeżeli pozwalają na to zasoby obliczeniowe, lub zredukowanej do maksymalnej, możliwej, rozsądnie wysokiej liczby wymiarów. Pozwala to na ograniczanie strat informacji wynikających z faktu redukcji wymiarowości.

```
In [8]: # Zdefiniowanie rozmiarów wykresu
plt.figure(figsize=(15,8))

# Opisujemy osie
plt.xlabel('1. składowa PCA')
plt.ylabel('2. składowa PCA')

# Narysujemy wykres punktowy. Rysujemy każdą klasę z osobna w pętli. Iterujemy po w
# wartościach występujących w zmiennej target.
for targ_val in np.unique(target):
    # Generujemy sobie maskę za pomocą której wybierzemy punkty należące do każdej
    mask = target==targ_val
    # Dla tak wszystkich punktów wybranych za pomocą odpowiadającej danej klasie ma
    plt.scatter(reduced_data[mask,0],reduced_data[mask,1], label = target_names[tar

# Na koniec dodajemy tytuł, legendę i rysujemy siatkę.
plt.title('Wizualizacja za pomocą 2 pierwszych składowych PCA')
plt.legend()
plt.grid()
```



Na wykresie także widoczny jest fakt, że to klasa setosa jest najlepiej separującą się klasą danych. Problemów z separacją możemy się spodziewać w przypadku klas versicolor i

virginica, gdyż także w przypadku wizualizacji bazującej na algorytmie PCA klasy te są umiejscowione blisko siebie.

Klasyfikacja nadzorowana bazująca na metryce odległości euklidesowej (algorytm k najbliższych sąsiadów)

Baując na danych pozyskanych za pomocą algorytmu PCA zwizualizujemy sobie zasadę działania algorytmu kNN. Będziemy dokonywać klasyfikacji na danych zredukowanych.

W praktyce unikalibyśmy takiego podejścia, bo tracimy informację zawartą w pełnych 4 składowych zbioru Iris. Na potrzeby ćwiczenia jednak taki zabieg pozwoli nam na łatwe i intuicyjne zwizualizowanie granic decyzyjnych wygenerowanych przez algorytm, który klasyfikacji dokonuje poprzez zastosowanie następujących kroków:

1.
Dla klasyfikowanego punktu danych określ odległości do k najbliższych punktów danych, dla których klasa jest znana
2.
Oblicz częstość występowania poszczególnych klas wśród k najbliższych punktów danych, dla których klasa jest znana
3.
punkt powinien być zakwalifikowany do tej klasy, która jest najczęstsza wśród klas k najbliższych punktów wyznaczonych w kroku poprzednim.

Efektem takiego postępowania jest utworzenie klasyfikatora charakteryzującego się krzywymi decyzyjnymi takimi, jak na rysunku będącym efektem wykonania kodu poniżej:

```
In [9]: # Wybierzmy sobie wartość k, którą przechowywać będziemy w zmiennej o opisowej nazwie
n_neighbors = 5

# Tworzymy klasyfikator dla wybranej liczby najbliższych sąsiadów.
kNN_classifier = KNeighborsClassifier(n_neighbors=n_neighbors)

# Trenujemy klasyfikator dla otrzymanych danych i ich etykiet (klasyfikacja nadzorzona)
kNN_classifier.fit(reduced_data, target)

# Rysujemy wykres - najpierw definicja rozmiarów
plt.figure(figsize=(15,8))

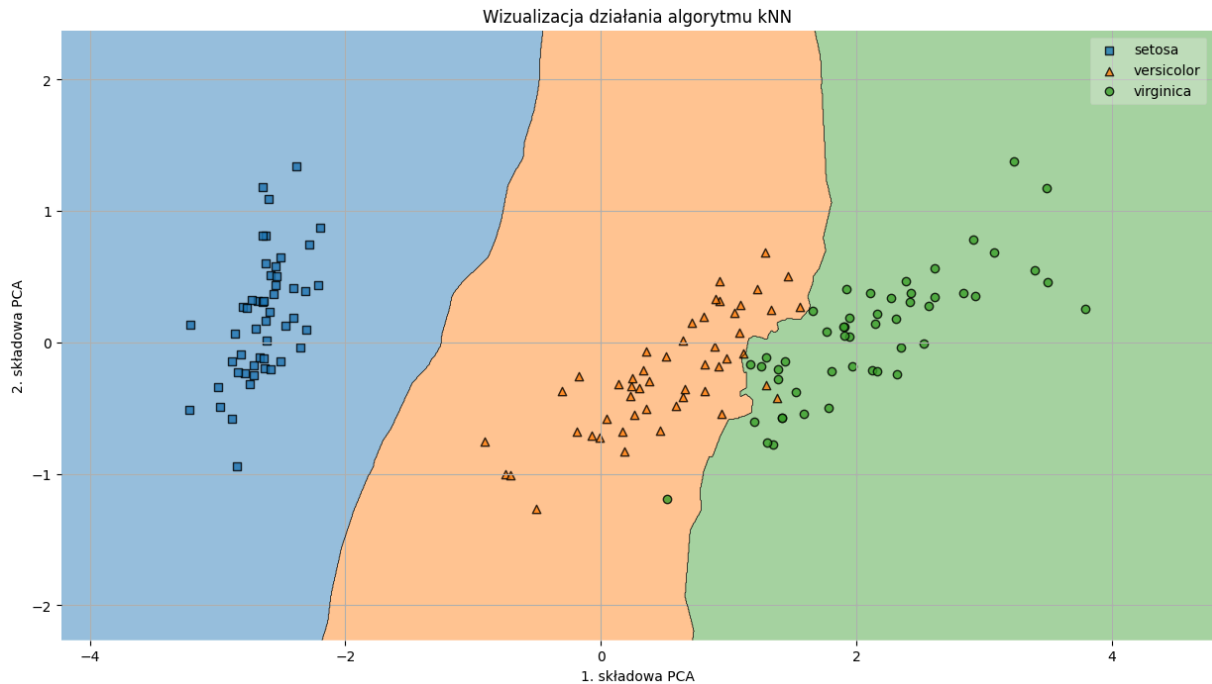
# Następnie wykorzystujemy funkcję z pakietu mlxtend, która pozwala na wygodne narysowanie
# granic decyzyjnych klasyfikatora
ax = plot_decision_regions(reduced_data, target, clf=kNN_classifier)

# Opisujemy osie, dodajemy tytuł i siatkę.
plt.xlabel('1. składowa PCA')
plt.ylabel('2. składowa PCA')
plt.title('Wizualizacja działania algorytmu kNN')
```

```
plt.grid()

# Na koniec zmodyfikujemy jeszcze legendę. Bez tego podane w niej są jedynie indeksy
# opisowe nazwy. Dla czytelności zmienimy je za pomocą poniższych linii na nazwy
# w zmiennej target_names.
handles, labels = ax.get_legend_handles_labels()
ax.legend(handles, target_names, framealpha=0.3, scatterpoints=1)
```

Out[9]: <matplotlib.legend.Legend at 0x201798d0e10>



Klasyfikacja nienadzorowana bazująca na metryce odległości euklidesowej (algorytm k średnich)

Możemy nieco zmodyfikować rozumowanie wykorzystywane do zdefiniowania algorytmu k najbliższych sąsiadów i dostosować je do sytuacji, w której nie wiemy dokładnie, jak punkty są przydzielone do klas. Nadal wymagana będzie wiedza o tym, ile klas zawiera się w zbiorze danych.

Algorytm k średnich jest (podobnie jak algorytm kNN) możliwy do opisania w postaci krótkiej listy czynności. Na początku musimy przyjąć, jakiej liczby klastrów spodziewamy się w analizowanych danych (teraz to właśnie ją oznaczmy jako k).

1.

Wybierz losowo k punktów ze zbioru danych i wykorzystaj je jako początkowe centra klastrów. W praktyce często wykorzystywana jest tutaj metoda `kmeans++`, która pozwala na wybranie punktów w sposób możliwie zwiększający zbieżność algorytmu k średnich. Więcej szczegółów można przeczytać np. w publikacji [D. Arthura i S. Vassilvitskiego](#), p.t. "k-means++: The Advantages of Careful Seeding".

2. Wykorzystaj tak wygenerowane centra klastrów by przypisać wszystkie punkty w zbiorze do klastra, którego centrum (zwane czasami centroidem) jest najbliższe. Czynność wykonywaną w ramach tego podpunktu jest tożsama z wykonaniem algorytmu kNN z liczbą sąsiadów równą 1 i centrami klastrów służącymi jako dane treningowe (najbliższe centrum decyduje o przynależności do klastra).
3. Na podstawie tak wygenerowanych klastrów wylicza się nowe ich centra. Wystarczy do tego uśrednić współrzędne punktów w każdym klastrze.
4. Jeżeli zmiana położenia klastrów jest mniejsza niż wybrany przez nas próg, np. 10^{-4} , to przerywamy wykonanie algorytmu, podobnie jeżeli osiągnęliśmy maksymalną liczbę iteracji (np. 100).
5. Jeżeli nie przerwaliśmy wykonania algorytmu w poprzednim kroku to wracamy do punktu 2..

Można zatem powiedzieć, że logika działania algorytmu kNN jest podstawą dla algorytmu k średnich. Przykład jego użycia na aktualnie przetwarzanym zbiorze danych widoczny jest poniżej:

```
In [10]: # Na początek zdefiniujemy kilka nastaw pracy algorytmu:
n_clusters      = 3      # Liczba klastrów, której się spodziewamy
n_init          = 1      # Scikit-Learn pozwala wykonać algorytm kilkakrotnie i wyb
                        # tutaj definiujemy ile razy obliczenia mają być powtórzon
num_iterations   = 100    # maksymalny limit iteracji algorytmu k średnich
tolerance       = 0.0001 # minimalna wartość, poniżej której uznajemy, że zmiany w

# Definiujemy algorytm k średnich i przekazujemy nastawy.
# Zmienną random_state ustawiamy na stałą wartość, by każde wykonanie algorytmu koń
# W innym przypadku generator losowości zawsze będzie startował z losowym ziarnem i
# różne wyniki po każdym jego ponownym uruchomieniu.
# Aby nie ustawiać generatora wartości losowych zawsze na tę samą wartość należy ni
clusterer = KMeans(n_clusters=n_clusters, n_init=n_init, max_iter=num_iterations, t

# Trenujemy algorytm na danych wejściowych
clusterer.fit(reduced_data)

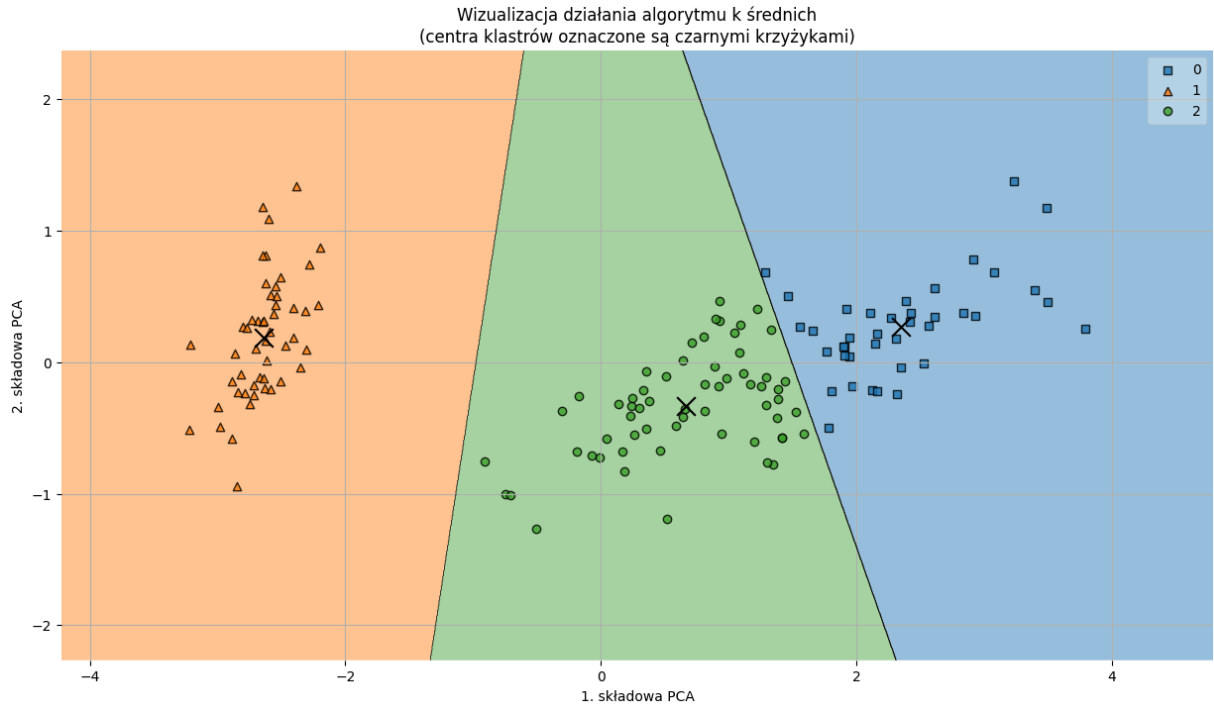
# Zapisujemy sobie wynik przypisania danych do klastrów.
all_predictions = clusterer.predict(reduced_data)

# Na potrzeby wizualizacji pobieramy także informację o położeniu centrów klastrów
centroids = clusterer.cluster_centers_

# Wizualizacja za pomocą wcześniej wprowadzonej już funkcji plot_decision_regions.
plt.figure(figsize=(15,8))
plot_decision_regions(reduced_data, all_predictions, clf=clusterer)
plt.scatter(centroids[:,0],centroids[:,1], c='black', marker='x', s=180)
plt.xlabel('1. składowa PCA')
plt.ylabel('2. składowa PCA')
```

```
plt.title("Wizualizacja działania algorytmu k średnich\n(centra klastrów oznaczone\nplt.grid())

# Aby zobaczyć różnicę pomiędzy różnymi liczbami iteracji czasami warto zapisać sob
# na dysku, aby potem szybko można było się przetaczać pomiędzy przypadkami dla róż
# (odkomentować linię w razie potrzeby)
# plt.savefig(f'kmeans_it_{str(num_iterations)}.zfill(3)}.png')
```



W praktyce warto zaznaczyć, że czasami (zwłaszcza dla zbiorów o bardzo dużej liczbie wymiarów) nie posługujemy się w takich analizach odległością Euklidesową. Okazuje się bowiem, że w takim przypadku zależności pomiędzy położeniem poszczególnych punktów łatwiej daje się uchwycić [za pomocą innych miar](#), np. miary L1 (Manhattan) czy tzw. ułamkowych miar odległości. Ma to związek z tzw. [klątwą wymiarowości](#).

Zadania do wykonania

Zadanie 1.

Uruchom algorytm k najbliższych sąsiadów dla kilku (około 3-4) wybranych wartości parametru k (rozpoczynając od 1).

1. Jaki charakter mają zmiany krzywej decyzyjnej wraz ze wzrostem tego parametru?
2. Czy dobór wartości tego parametru może potencjalnie wpływać na generalizację klasyfikatora kNN?

Zadanie 2.

Uruchom algorytm k średnich i zbadaj, jak wyglądają linie podziału biegnące pomiędzy klastrami dla różnych wartości parametru k . Zapisz też na dysku (za pomocą zakomentowanej linijki, dla $k=3$) wizualizacje efektów działania algorytmu dla kilku pierwszych iteracji algorytmu k średnich (zmieniając wartość zmiennej `num_iterations` np. od 1 do 5).

1.

Opisz, jaki charakter mają zmiany następujące w położeniu klastrów (z iteracji na iterację)? W sprawozdaniu możesz, ale nie musisz zawierać ilustracji graficznej tych zmian (np. w postaci zrzutów ekranu).

2.

Czy algorytm dobrze poradził sobie z "odgadnięciem" podziału na grupy jeżeli przyjmemy, że $k=3$? Odpowiedź uzasadnij.

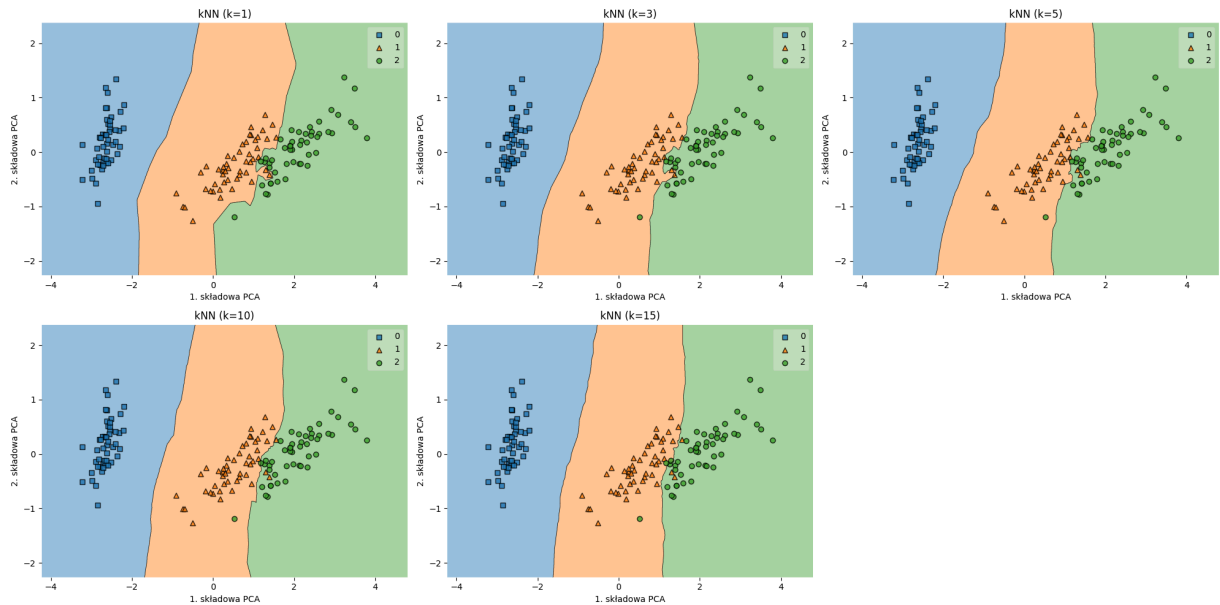
```
In [11]: # Zadanie 1 - Testowanie algorytmu kNN dla różnych wartości k
k_values = [1, 3, 5, 10, 15]

plt.figure(figsize=(20, 10))

for i, k in enumerate(k_values, start=1):
    plt.subplot(2, 3, i)
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(reduced_data, target)

    plot_decision_regions(reduced_data, target, clf=knn)
    plt.title(f'kNN (k={k})')
    plt.xlabel('1. składowa PCA')
    plt.ylabel('2. składowa PCA')

plt.tight_layout()
plt.show()
```



Wnioski - Zadanie 1:

1. Wraz ze wzrostem parametru k , granice decyzyjne stają się coraz bardziej wygładzone i uproszczone. Dla małych wartości (np. $k=1$) granica jest bardzo poszarpana ("ząbkowana"), ściśle dopasowując się do poszczególnych punktów uczących. Z kolei dla większych k (np. 10 lub 15), skrajne pojedyncze obserwacje są ignorowane (traktowane jak szum), a granice mają bardziej regularny kształt.
2. Tak, dobór parametru k wpływa na generalizację. Jeśli jest zbyt małe ($k=1$), model ma skłonność do przeuczenia (overfitting) – uczy się "szumu". Kiedy system operuje na większych k , model zyskuje lepszą zdolność do generalizacji (underfitting zaczyna się, gdy k jest zbyt duże i granice stają się za proste z pominięciem naturalnych wzorców). Trzeba znaleźć optymalny balans.

```
In [12]: # Zadanie 2 - Różne wartości parametru k
k_values_kmeans = [2, 3, 4, 5]

plt.figure(figsize=(20, 5))

for i, k in enumerate(k_values_kmeans, start=1):
    plt.subplot(1, 4, i)
    clusterer = KMeans(n_clusters=k, n_init=10, random_state=42)
    clusterer.fit(reduced_data)
    predictions = clusterer.predict(reduced_data)
    centroids = clusterer.cluster_centers_

    plot_decision_regions(reduced_data, predictions, clf=clusterer)
    plt.scatter(centroids[:,0], centroids[:,1], c='black', marker='x', s=180)
    plt.title(f'k-Means (k={k})')
    plt.xlabel('1. składowa PCA')
    plt.ylabel('2. składowa PCA')

plt.tight_layout()
plt.show()
```



```
# Zbadanie pierwszych iteracji dla k=3 i zapis na dysku
plt.figure(figsize=(25, 5))
for i, iters in enumerate(range(1, 6), start=1):
    plt.subplot(1, 5, i)
    clusterer = KMeans(n_clusters=3, n_init=1, max_iter=iters, tol=0.0, random_state=0)
    clusterer.fit(reduced_data)
    predictions = clusterer.predict(reduced_data)
    centroids = clusterer.cluster_centers_

    plot_decision_regions(reduced_data, predictions, clf=clusterer)
    plt.scatter(centroids[:,0], centroids[:,1], c='black', marker='x', s=180)
    plt.title(f'k-Means iter={iters}')
    plt.xlabel('1. składowa PCA')
    plt.ylabel('2. składowa PCA')

# Zapis wizualizacji dla pierwszych iteracji
plt.savefig(f'kmeans_it_{str(iters).zfill(3)}.png')

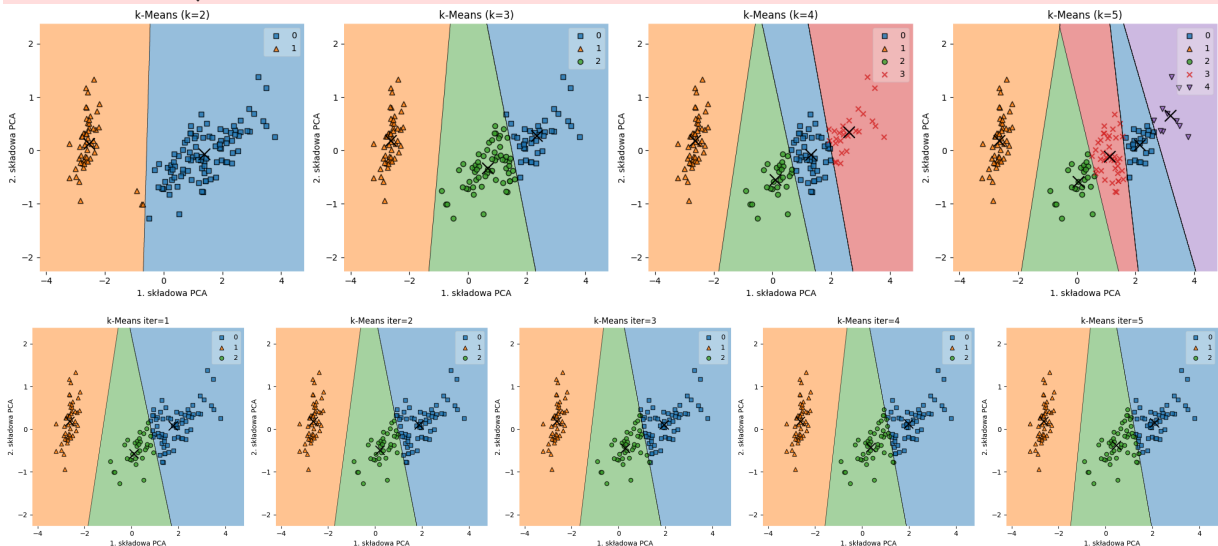
plt.tight_layout()
plt.show()
```

c:\Users\Igor\anaconda3\Lib\site-packages\mlxtend\plotting\decision_regions.py:346: UserWarning: You passed a edgecolor/edgecolors ('black') for an unfilled marker ('x'). Matplotlib is ignoring the edgecolor in favor of the facecolor. This behavior may change in the future.

```
ax.scatter(
```

c:\Users\Igor\anaconda3\Lib\site-packages\mlxtend\plotting\decision_regions.py:346: UserWarning: You passed a edgecolor/edgecolors ('black') for an unfilled marker ('x'). Matplotlib is ignoring the edgecolor in favor of the facecolor. This behavior may change in the future.

```
ax.scatter(
```



Wnioski - Zadanie 2:

1. Analizując kolejne iteracje algorytmu dla $k=3$, widać jak centroidy z początkowych, często suboptymalnych pozycji sukcesywnie przesuwają się w stronę "środków ciężkości" każdej z grup punktów. Wraz z przemieszczaniem się centroidów, linie podziału (granice decyzyjne) również systematycznie się dostosowują, dążąc do

stabilizacji. Zmiany te są najbardziej drastyczne w pierwszych 1-2 iteracjach, a później algorytm jedynie nieznacznie "dostraja" granice.

2. Algorytm dobrze radzi sobie z wyodrębnieniem w 100% klasy (Setosa), ponieważ odstaje ona znacznie od reszty zbioru. Natomiast dla klas Versicolor i Virginica, które w zredukowanej (jak i pełnej) przestrzeni silnie się przenikają, algorytm k-Means sztucznie kroi chmurę punktów w połowie za pomocą w miarę "sztywnej" linii, opierając się jedynie na odległości euklidesowej. Powoduje to, że punkty leżące na pograniczu tych klas często łądąją w niewłaściwym, sąsiednim klastrze. Podział nie jest więc w tym miejscu idealnie wierny rzeczywistej przynależności klasowej.



Rzeczpospolita
Polska

Unia Europejska
Europejski Fundusz
Rozwoju Regionalnego

